

****UNIT – I**

Introduction to Data Structures (Complete, Student-Friendly Notes)**

1. What Are Data Structures?

A **Data Structure** is a way of *storing, organizing, and managing data* so that it can be used efficiently.

Why do we need Data Structures?

- To store data in a structured way
 - To perform operations like searching, insertion, deletion efficiently
 - To handle large amounts of data
 - To improve program performance
 - To reduce memory wastage
-

2. Elementary Data Structures

These are the fundamental structures used in most programs.

2.1 Stacks

A **Stack** is a linear data structure that follows **LIFO (Last In First Out)** order.

Real-Life Examples

- Stack of plates
- Undo operation in editors
- Browser back button

Operations

Operation	Meaning
<code>push (x)</code>	Insert an element
<code>pop ()</code>	Remove the top element

Operation	Meaning
<code>peek()</code>	See the top element
<code>isEmpty()</code>	Check if stack is empty

Diagram

Top → 30
 20
 10

2.2 Queues

A **Queue** is a linear structure that follows **FIFO (First In First Out)**.

Real-Life Examples

- People standing in line
- Print queue
- Task scheduling

Operations

Operation	Meaning
<code>enqueue(x)</code>	Add element at rear
<code>dequeue()</code>	Remove element from front
<code>peek()</code>	View front element
<code>isEmpty()</code>	Check if queue is empty

Diagram

Front → [10, 20, 30] ← Rear

2.3 Linked Lists

A **Linked List** is a sequence of nodes where each node contains:

- **Data**
- **Pointer (reference) to next node**

Types

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

Advantages

- Dynamic size
- Easy insertion/deletion

Applications

- Implementing stacks & queues
- Memory management
- Graph representation

Node Structure

[data | next] → [data | next] → ...

3. Implementing Pointers and Objects

Pointers

A pointer stores the *address* of another variable or node.
In linked lists, pointers connect nodes.

Objects (in OOP)

Objects store data and member functions.
Eg: Node can be an object containing `data` and `next`.

4. Representing Rooted Trees

A **Tree** is a hierarchical data structure.

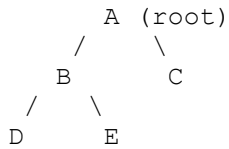
Key Terms

- **Root** – topmost node
- **Parent/Child** – relationship
- **Leaf** – node with no children
- **Depth** – levels from the root

Representation Methods

1. **Linked Structure (using nodes & pointers)**
2. **Array Representation**

Example Tree



5. Hash Tables

A **Hash Table** stores key–value pairs for **very fast searching**.

5.1 Direct Address Tables

Uses the key itself as an index.

Example:

If student IDs range from 0–999, store records directly at `array[id]`.

Advantages

- $O(1)$ lookup

Disadvantage

- Needs large memory
-

5.2 Hash Tables

A hash table uses a **hash function** to convert a key into an index.

Hash Function

`index = h(key)`

Good Hash Function Properties

- Fast
 - Distributes keys uniformly
 - Minimizes collisions
-

5.3 Collisions

When two keys map to the same index.

This is resolved using **Collision Resolution Techniques**.

6. Collision Resolution Techniques (Open Addressing)

Open Addressing

If one index is full, find another empty index.

Methods

1. **Linear Probing**
 - $\text{index} = (\text{h}(\text{key}) + i) \% \text{size}$
 2. **Quadratic Probing**
 - $\text{index} = (\text{h}(\text{key}) + i^2) \% \text{size}$
 3. **Double Hashing**
 - $\text{index} = (\text{h1}(\text{key}) + i * \text{h2}(\text{key})) \% \text{size}$
-

7. Binary Search Trees (BST)

A **Binary Search Tree** is a binary tree with rules:

- Left child < Root
- Right child > Root

Example



Advantages

- Fast searching
 - Keys always kept sorted
-

8. Querying a Binary Search Tree

Searching

1. Compare key with root
2. If key < root → move left
3. If key > root → move right
4. If equal → found

Time Complexity:

- Best: $O(\log n)$
 - Worst: $O(n)$ (skewed tree)
-

9. Insertion in BST

Steps:

1. Compare value with root
 2. If smaller → go left
 3. If greater → go right
 4. Insert when empty position is found
-

10. Deletion in BST

Three cases:

- ✓ **Case 1: Node has no children (leaf)**

Just delete.

✔ Case 2: Node has one child

Replace node with its child.

✔ Case 3: Node has two children

Replace with:

- **Inorder successor** (smallest value in right subtree) OR
- **Inorder predecessor**

Complete Summary / Mind Map

Data Structures

